

Chapter 2 _ Understanding Foundation Models

Domain Specific Models

A general-purpose model can perform on various domains quite well, but not for a specific domain like coding, laws. This is where the Specific domain sheds its light.

Two examples of domain-specific models are Drug discovery and cancer screening.

Modeling

Before training a model, the developers have to decide what the model should look like and what architecture it should follow. How many parameters should it have? These decisions impact not only the model's capabilities but also its usability for downstream applications. For example, the 7-B parameters model may be easily deployed compared to the larger parameter model. Similarly, optimizing a transformer model for latency is very different from optimizing another architecture.

Model Architecture

The most dominant architecture for language-based foundation models is the transformer architecture.

Transformer Architecture

The transformer architecture was popularized on the heels of the Seq2Seq architecture.

Seq2Seq contains an encoder that processes inputs and a decoder that generates outputs. They are sequences of tokens.

- In a transformer, the input is embedded into a continuous vector for each individual token (word or sub-word). However, when looking at a sequence of text, these individual vectors are stacked together to form a matrix.

The **encoder** processes the input tokens sequentially, outputting the final [Hidden state](#) that represents the input.

The **decoder** then generates output tokens sequentially, conditioned on both the final hidden state of the input and the previously generated token.

However, there are two problems with the Seq2Seq:

- The decoder generates output using only the final hidden state of the input. (Generating answers about a book by using a book) --> This limits the quality of the output.
- The RNN encoder and decoder, meaning that the outputs are generated and the inputs are processed sequentially, making it slow for long sequences.
- So the transformer architecture addresses these two problems with the attention mechanism.
- **Self-Attention mechanism** allows token in a sequence to directly attend to every other token, capturing relationships regardless of their distance.

- The **attention mechanism** allows the model to weight the importance of different input tokens when generating each output token.

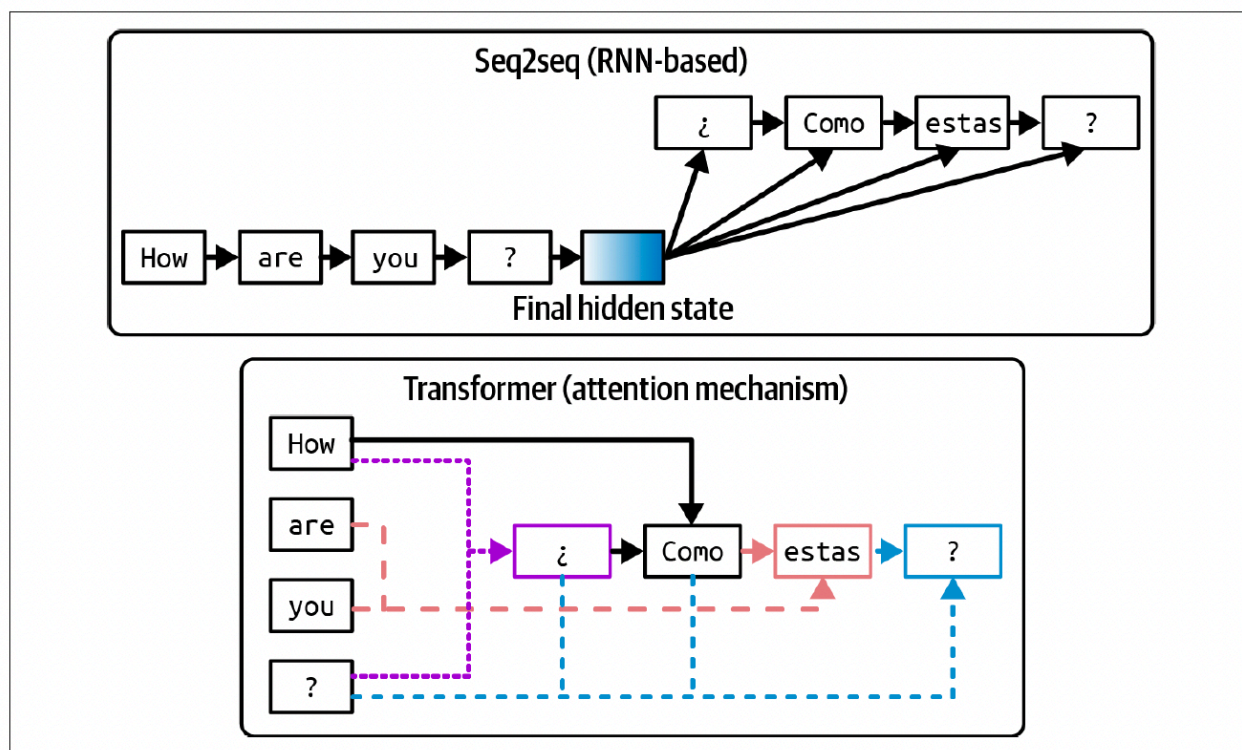


Figure 2-4. Seq2seq architecture versus transformer architecture. For the transformer architecture, the arrows show the tokens that the decoder attends to when generating each output token.

- With a transformer architecture, the input tokens can be processed in parallel, speeding up the input process.
- The **attention mechanism** leverages **key**, **value**, and **query vectors**:
- The query vector (Q) represents the current state of the decoder at each decoding step. This query vector can be thought of as the person looking for information to create a summary
- The Key vector (K) represents a previous token. If each previous token is a page in a book, each key vector is like the page number. **Note:** In the given decoding step, previous tokens include both input tokens and previously generated tokens.
- Each value vector (V) represents the actual value of a previous token, as learned by the model. Each value vector is like the page's content

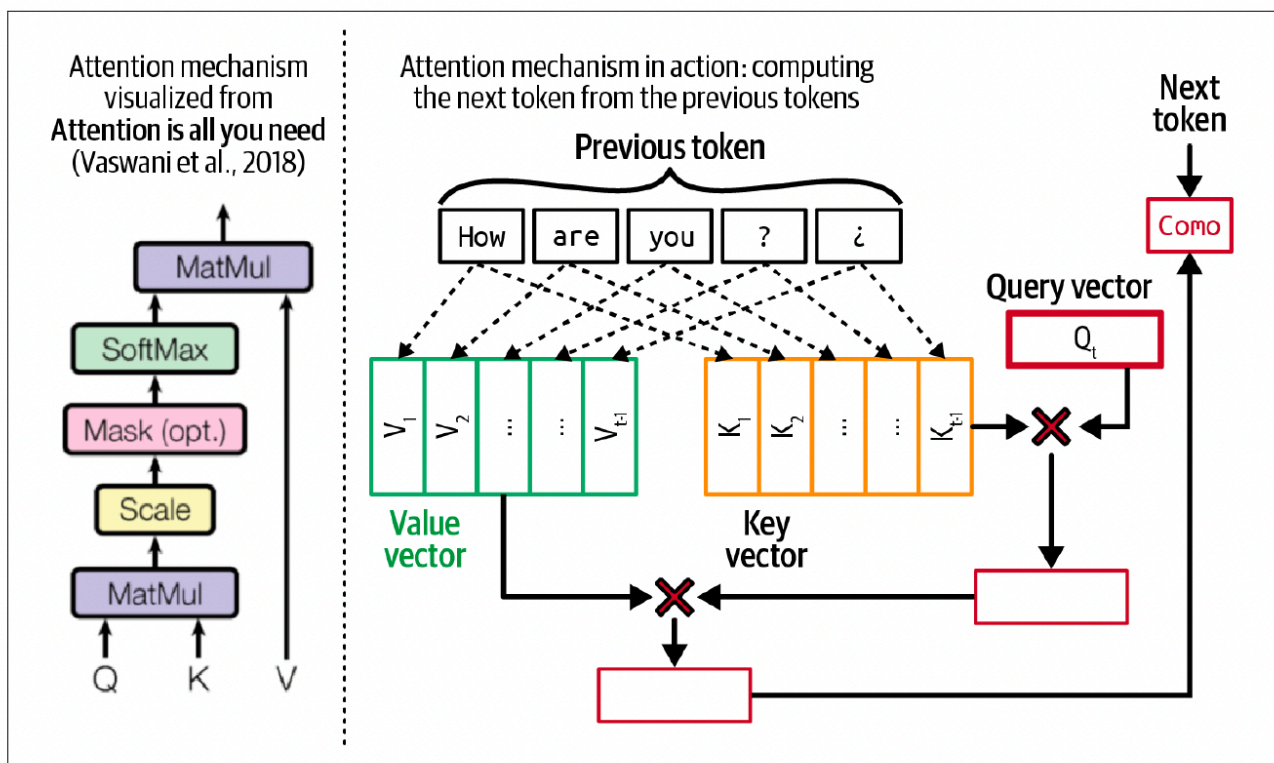


Figure 2-5. An example of the attention mechanism in action next to its high-level visualization from the famous transformer paper, “Attention Is All You Need” (Vaswani et al., 2017).

- How the attention mechanism works:
- Given an input X , the key vector, value vector, and query vector are computed by applying the key, value, and query matrices to the input. Let K, V, Q be the key, value, and query matrices. They are computed as follows:
 - $K = X \cdot W_K$
 - $V = X \cdot W_V$
 - $Q = X \cdot W_Q$
 And these three matrices have dimensions corresponding to the model's hidden dimension. For example: Llama 2-7B has hidden dimension size is 4096, meaning that each of these matrices has a 4096x4096 dimensions. Each resulting K, V, Q vector has the dimension of 4096.
- The Attention mechanism is always multi-headed. **The multi-heads** allows the model to attend different groups of previous tokens simultaneously. and with the multi-headed attention, the query, key, and value vectors are split into smaller vectors, each corresponding to an attention head. For example: Llama 2-7B has 32 attention heads, so each K, V , and Q vector will be split into 32 vectors of the dimension 128, simply because $4096/32 = 128$ -
- **Attention score** = the dot product of Q and K , measuring relevance between tokens.
- **Transformer Block:** A transformer architecture consists of multiple transformer blocks. The exact content of the block varies between models, but, in general, each transformer block contains the attention module and the MLP (Multi-Layer Perceptron) module:
- **Attention Module:**
 - Each attention module consists of four weight matrices: **query, key, value, and output projection**
- **MLP Module:**
 - Consist of linear layers separated by non-linear activation functions.
 - Each linear layer is a weight matrix that is used for linear transformations, whereas an activation function allows the linear layers to learn non-linear patterns.

- A linear layer is also called a feedforward layer.
- The number of transformer blocks = Model's number of layers
- A transformer-based language model is also outfitted with a module before and after all the transformer blocks:
- An embedding module before the transformer blocks
- This module consists of the embedding matrix and positional embedding matrix, which convert tokens and their positions into embedding vectors, respectively. Naively, the number of position indices determines the model's maximum context length. For example, if the model keeps track of 2048 positions, its maximum context's length is 2048.
- An output layer after the transformer blocks:
- the module maps the model's output vectors into token probabilities used to sample model outputs. This module typically consists of one matrix, which is also called the unembedding layer. Some people refer to the output layer as the model head, as it is the model's last layer before output the generation.

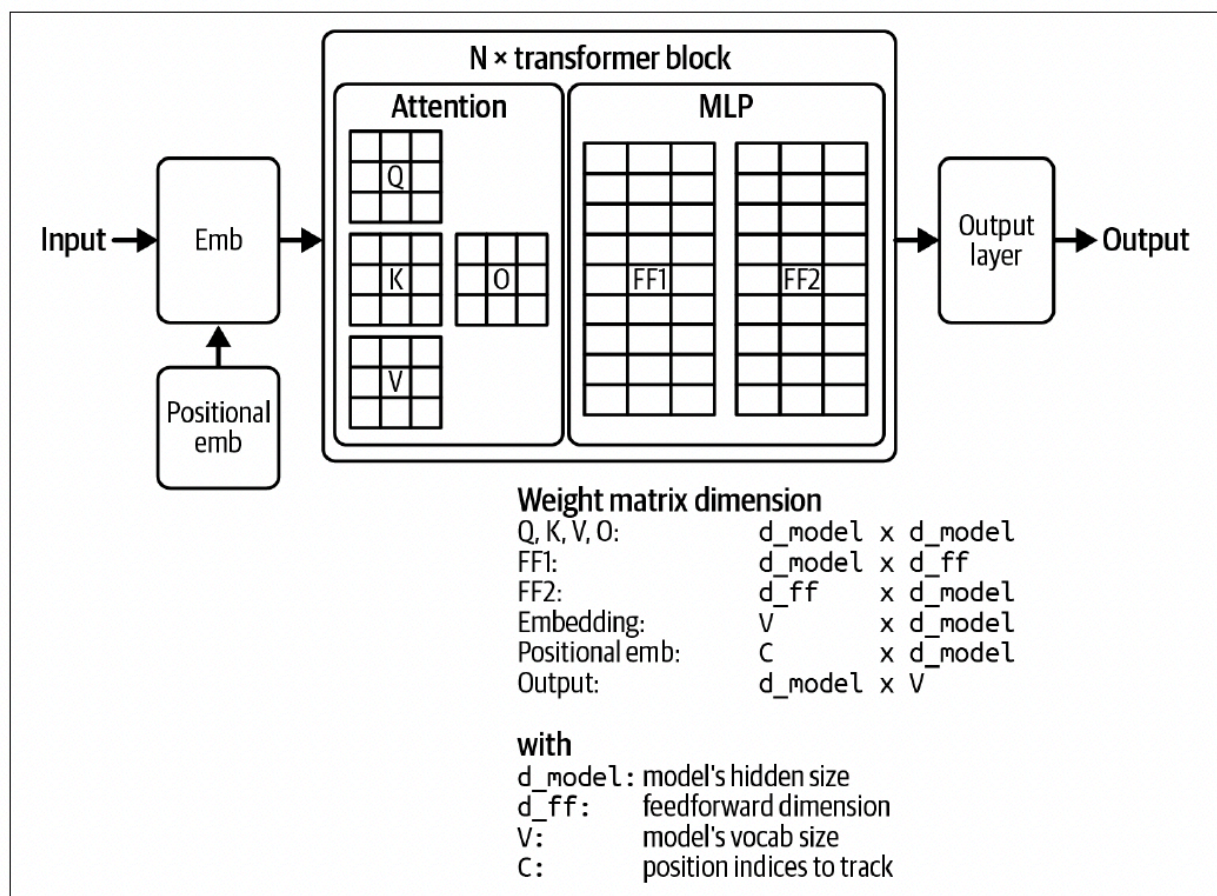


Figure 2-6. A visualization of the weight composition of a transformer model.

And, the backpropagation is performed during the training.

During the training:

Input → Transformer → Prediction → Loss

Then:

Loss → Backpropagation → Update the weights

The model computes:

- Prediction error
 - Gradients
 - Parameter updates
- Using:
- Gradient Descent
 - Adam Optimizer
 - Backpropagation

So, for the transformer architecture:
Gradients flow backward through"

1. Output layer
2. MLP layers
3. Attention layers
4. Embeddings

Other Model Architectures

RWKV, it is an RNN-based model that can be parallelized for training. Due to its RNN nature, in theory, it does not have the same context length limitation that transformer-based models have. However, in practice, having no context length limitation does not guarantee good performance with long context.

Model Size

The number of parameters is usually appended at the end of a model name.

The number of parameters also helps us estimate the compute resources needed to train and run this model.

For example, if a model has 7 billion parameters, and each parameter is stored using 2 bytes (16 bits), then we can calculate that the GPU memory needed to do inference using this model will be at least 14 billion bytes (14 GB)

- Also, the number of parameters can be misleading if the model is sparse. A sparse model has a large percentage of zero-value parameters.
 - A 7B-parameter model that is 90% sparse only has 700 million non-zero parameters. Sparsity allows for more efficient data storage and computation.
 - This mean that a large sparse model can require less compute than a small dense model.
- A larger model can also underperform a smaller model if it is not trained on enough data. Imagine a 13B-param model trained on a dataset consisting of a single sentence "I like pineapples." this model will perform much worse than a much smaller model trained on more data.

Token is the unit that a model operated on, knowing the number of tokens in a dataset helps us measure how much a model can potentially learn from that data.

The number of tokens in a model's dataset is not the same as its number of training tokens. The number of training tokens measure the tokens that the model is trained on. If a dataset contains 1 trillion tokens and a model is trained on that datasets for two epochs- - an epoch is a pass through the dataset--the number of training tokens is 2 trillion.

A more standardize unit for the model's compute requirement is FLOP (Floating Point operation). It measures the number of floating point operations performed for a certain task. Google's largest PaLM-2 model was trained using FLOPs

Note: FLOPs measure the compute requirement for a task, whereas FLOP/s measures a machine's peak performance.

All in all:

- The parameter = a proxy for the model's learning capacity
- The number of tokens a model was trained on = a proxy for how much a model learned
- Number of FLOPs = a proxy for the training cost

Scaling law

Given a fixed amount of FLOPs, what model size and dataset size would give the best performance? A model that can achieve the best performance given a fixed compute budget is compute-optimal.

For a compute-optimal model size, we need the number of training token approximately 20 times the number of tokens, meaning that a 3B-param model needs approximately 60B training tokens. The model size and the number of training tokens should be scaled equally: for every doubling of the model size, the number of training tokens should also be doubled.

The compute-optimal calculation assumes that the cost of acquiring data is much cheaper than the cost of the compute.

Scaling Extrapolation

The performance of the model depends predominantly on the number of hyperparameters.

Scaling extrapolation (hyperparameter transfer) aims to predict, for large models, which hyperparameters will yield the best performance.

it is still a niche topic, as few people have the experience and resources to study the training of large models. It is also difficult due to the sheer number of hyperparameters and their interactions. if you have ten hyperparameters, you'd have to study 1024 hyperparameter combinations. You would have to study each hyperparameter individually, then two of them together, and three of them together, and so on.

Scaling bottleneck

Training data and electricity.

The rate of training dataset size growth is much faster than the rate of new data being generated

If you have ever put anything on the internet, you should assume that it already is or will be included in the training data for some language models, whether with your consent or not. This is similar to how, if you post something on the internet, you should expect it to be indexed by Google.

If the company continues using the training data from the internet, these new models will likely be trained partially on AI-generated data. It is a worry that recursively training new AI models on AI-generated data causes the new models to gradually forget the original data patterns, degrading their performance over time.

And the data center of the model is expected to consume 2% of the global electricity. The number is estimated to be between 4% and 20% by 2030.

Post-Training

Post-training consists of two steps:

- **Supervised Finetuning (SFT):** Finetuning the pre-trained model on high-quality instruction data to optimize the models for conversations instead of completion.
 - To encourage a model to generate the appropriate responses, we can show examples of appropriate responses. Such examples follow the format (prompt, response) and are called demonstrated data.
- **Preference Finetuning:** Further finetune the model to output responses that align with human preferences. Preference Finetuning is typically done with reinforcement learning. Techniques for preference Finetuning include reinforcement learning from human feedback (RLHF) (used by GPT-3.5 and Llama 2), DPO (Direct Preference Optimization) (Used by Llama 3), and reinforcement learning from AI feedback (RLAIF) (Used by Claude).
 - Demonstration data teaches the model to have a conversation but does not teach the model to have a conversation but does not teach the model what kind of conversations it should have.
 - If the model is censored too much, it becomes boring, driving the user.
 - The goal of the preference finetuning is to get the AI to behave according to human preferences.
 - RLHF:
 - Train a reward model that scores the foundation model's output
 - Optimize the foundation model to generate responses for which the reward model will maximize scores.

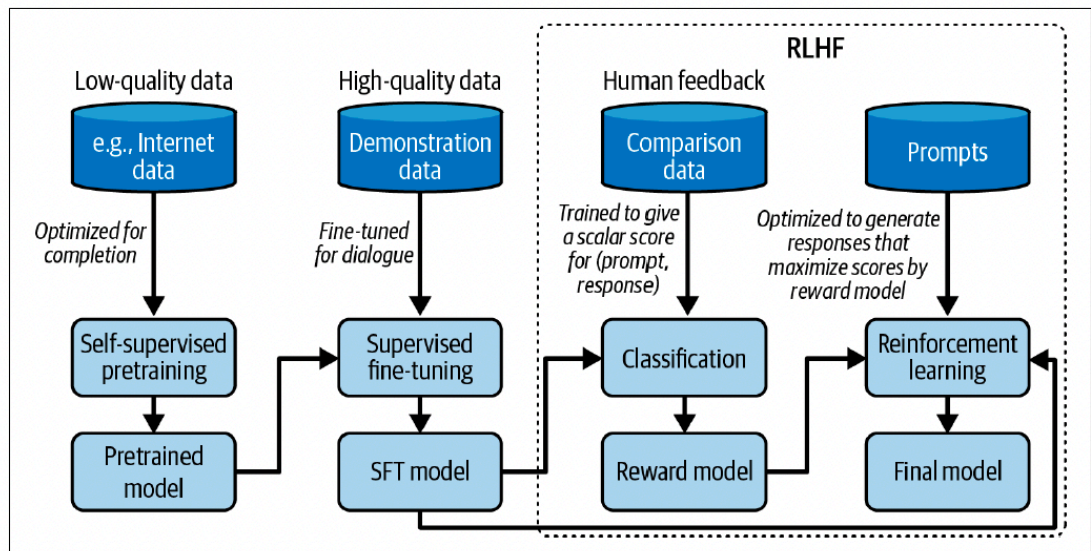


Figure 2-10. The overall training workflow with pre-training, SFT, and RLHF.

Reward Model: RLHF relies on the reward model. Given a pair of (Prompt, response), the reward model outputs a score for how good the response is.

- Evaluating each sample independently is also called "Pointwise Evaluation"

Sampling

Sampling is how a model picks out an output of all possible outputs.

The right sampling strategy can make a model generate responses more suitable for the application, like a more creative response... or make the generalization more predictable.

You can also design your own sampling strategy, though this typically requires access to the model's logits.

Common sampling strategies:

Temperature

A higher temperature reduces the probabilities of common tokens and, as a result, increases the probabilities of rarer tokens. This enables models to create more creative responses, but less coherent ones, and vice versa.

A temperature of 0.7 is often recommended for creative use cases, as it balances creativity and predictability, but doing your own experiment is also encouraged!

Many model providers return probabilities generated by their models as logprobs.

Logprobs, short for Log probabilities, are probabilities on the log scale. A log scale is preferred when working with nn probabilities because it helps mitigate underflow.

A language model might be working with a vocabulary size of 100,00, which means the probabilities for many of the tokens can be too small to be represented by a machine. The small numbers might be rounded down to 0. Log scale helps reduce this problem.

and the logprob values are usually negative, because the log of values between 0 and 1 is negative.

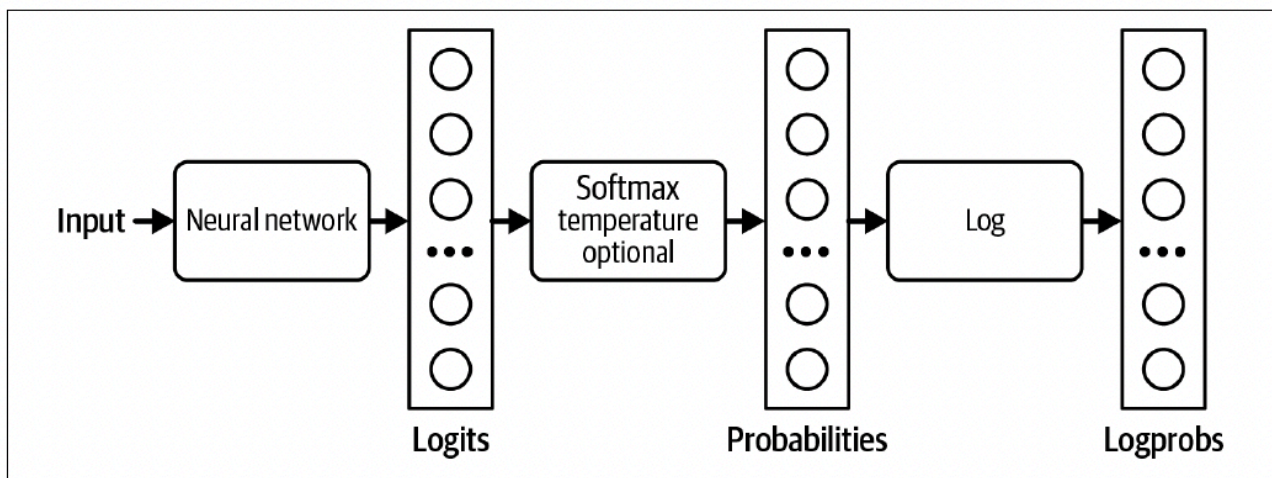


Figure 2-17. How logits, probabilities, and logprobs are computed.

Top K

It reduce the computation workload without sacrificing too much of the model's response diversity.

Recall that, the softmax layer is used to compute the probability distribution over all possible values. Softmax requires two pass all possible values:

- One to perform the exponential sum , and one to perform for each value

For a large language model with a large vocabulary size, this process is computationally expensive.

So to avoid this problem, after the model has computed the logits, we picks the top k logits and perform softmax over these top k logits only.

The top k can be anywhere between 50 to 500 -- much smaller than the model's vocabulary size.

Top P

it is also known as nucleus sampling, allows for a more dynamic selection of values to be sampled from. in top-p sampling, the model sums the probabilities of the most likely next values in descending order and stops when the sum reaches p. only the value within this cumulative probabilities are considered.

Common top p value: 0.90-0.95

A top p of 0.90 mean that the model will consider the smallest set of values whose cumulative probabilities exceed 90%

Stopping Condition

So, an autoregressive language model generates sequences of tokens by generating one token after another. a long output sequence takes more time, cost more compute, and can sometimes annoy the users. Thus, we might want to set the condition for the model to stop the sequence

Stopping condition is a method to ask the model to stop generating after a fixed number of tokens. it is beneficial to keep the latency and costs down.

Downside:

- premature stopping can cause the outputs to be malformed. JSON --> JSON to be missing the closing brackets, making the generated JSON hard to parse.

Test Time Compute

One simplest way to improve a model's response quality is test time compute: Instead of generating only one response per query, you generate multiple responses to increase the chance of good responses. And the simplest way to enhance the effectiveness of the time compute is to increase the diversity of the outputs, because a more diverse set of options is more likely to yield better candidate.

Keep on generating the outputs until one fits the expected format.

To pick the best output, you can either show users multiple outputs and let them choose the one that works best for them, or you can devise a method to select the best one. One selection method is to pick the output with the highest probabilities.

A language models' output is a sequence of tokens, and each token has a probability computed by the model. The probability of an output is the product of the probabilities of all tokens in the output.

Picking out the most common output among a set of outputs can be especially useful for tasks that expect the exact answers.

The less robust a model is, the more you can benefit from sampling multiple outputs.

Structured Outputs

Structured Outputs are crucial for the following two scenarios:

- Tasks requiring structured outputs:
- Tasks whose outputs are used by downstream applications: the outputs are used by other application, they need to be parsable by these applications.

The other four approaches

- **Prompting**
- **Post-processing:** simple and cheap. A model tend to make similar mistakes across queries, meaning that if you can find the common mistake a model makes, you can potentially write a script to correct it.
 - It works only if the mistake is easy to fix.
- **Constrained sampling:** a technique for guiding the generation of text toward certain constraints. it is typically followed by the structured output tools.

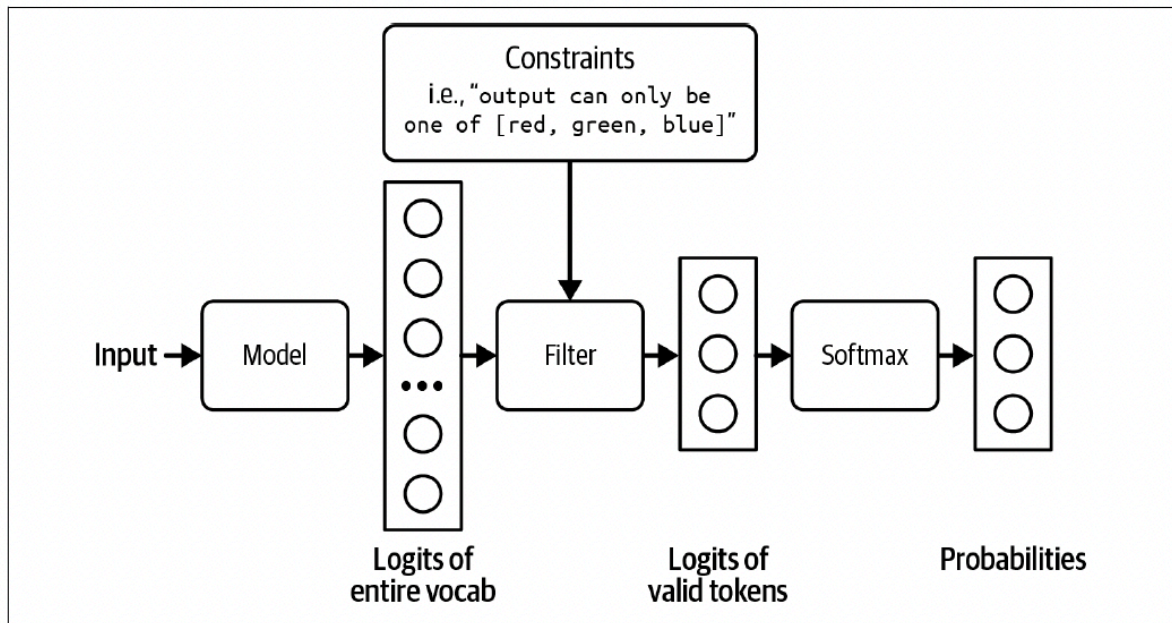


Figure 2-21. Filter out logits that don't meet the constraints in order to sample only among valid outputs.

- Finetuning: finetuning the model on the examples following your desirable format is the most effective and general approach to get the models to generate output in this format. While simple finetuning does not guarantee that the model will always output the expected format, it is much more reliable than prompting.
 - During the finetuning, we can also retrain the model end-to-end or part of the model, such as this classifier head.

The probabilistic nature of AI

- Inconsistency
- Hallucination